

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #define MAX_STRING_LENGTH 6
5
6  struct package
7  {
8      char* id;
9      int weight;
10 };
11
12 typedef struct package package;
13
14 struct post_office
15 {
16     int min_weight;
17     int max_weight;
18     package* packages;
19     int packages_count;
20 };
21
22 typedef struct post_office post_office;
23
24 struct town
25 {
```

```

24 struct town
25 {
26     char* name;
27     post_office* offices;
28     int offices_count;
29 };
30
31 typedef struct town town;
32
33
34
35 void print_all_packages(town t) {
36     printf("%s:\n",t.name);
37     for(int i=0;i<t.offices_count;i++)
38     {
39         printf("\t%d:\n", i);
40         for(int j=0;j<t.offices[i].packages_count;j++)
41         {
42             printf("\t\t%s\n",t.offices[i].packages[j].id);
43         }
44     }
45 }
46
47 void send_all_acceptable_packages(town* source, int source_office_index, town* target, int
target_office_index) {

```

```

7 void send_all_acceptable_packages(town* source, int source_office_index, town* target, int
target_office_index) {
8     post_office *src = &source->offices[source_office_index];
9     post_office *dst = &target->offices[target_office_index];
10
11     int accepted = 0;
12
13     for(int i = 0; i < src->packages_count; i++)
14     {
15         if(src->packages[i].weight >= dst->min_weight &&
16            src->packages[i].weight <= dst->max_weight)
17         {
18             accepted++;
19         }
20     }
21
22     int old_src_count = src->packages_count;
23     int old_dst_count = dst->packages_count;
24
25     package *new_src =
26         malloc(sizeof(package) * (old_src_count - accepted));
27
28     package *new_dst =

```

```
package *new_dst =  
    malloc(sizeof(package) * (old_dst_count + accepted));  
  
for(int i = 0; i < old_dst_count; i++)  
{  
    new_dst[i] = dst->packages[i];  
}  
  
int si = 0;  
int di = old_dst_count;  
  
for(int i = 0; i < old_src_count; i++)  
{  
    if(src->packages[i].weight >= dst->min_weight &&  
        src->packages[i].weight <= dst->max_weight)  
    {  
        new_dst[di++] = src->packages[i];  
    }  
    else  
    {  
        new_src[si++] = src->packages[i];  
    }  
}
```

```
free(src->packages);
free(dst->packages);

src->packages = new_src;
src->packages_count = old_src_count - accepted;

dst->packages = new_dst;
dst->packages_count = old_dst_count + accepted;
}
```

```
town town_with_most_packages(town* towns, int towns_count) {
    int max = -1;
    int index = 0;

    for(int i = 0; i < towns_count; i++)
    {
        int total = 0;

        for(int j = 0; j < towns[i].offices_count; j++)
        {
            total += towns[i].offices[j].packages_count;
        }
    }
}
```

```
        if(total > max)
        {
            max = total;
            index = i;
        }
    }

    return towns[index];
}
```

```
town* find_town(town* towns, int towns_count, char* name) {
    for(int i = 0; i < towns_count; i++)
    {
        if(strcmp(towns[i].name, name) == 0)
        {
            return &towns[i];
        }
    }

    return NULL;
}
```



```

int main()
{
    int towns_count;
    scanf("%d", &towns_count);
    town* towns = malloc(sizeof(town)*towns_count);
    for (int i = 0; i < towns_count; i++) {
        towns[i].name = malloc(sizeof(char) * MAX_STRING_LENGTH);
        scanf("%s", towns[i].name);
        scanf("%d", &towns[i].offices_count);
        towns[i].offices = malloc(sizeof(post_office)*towns[i].offices_count);
        for (int j = 0; j < towns[i].offices_count; j++) {
            scanf("%d%d%d", &towns[i].offices[j].packages_count, &towns[i].offices[j].min_weight,
            towns[i].offices[j].max_weight);
            towns[i].offices[j].packages = malloc(sizeof(package)*towns[i].offices[j].
            packages_count);
            for (int k = 0; k < towns[i].offices[j].packages_count; k++) {
                towns[i].offices[j].packages[k].id = malloc(sizeof(char) * MAX_STRING_LENGTH);
                scanf("%s", towns[i].offices[j].packages[k].id);
                scanf("%d", &towns[i].offices[j].packages[k].weight);
            }
        }
    }
}

```

```

}
int queries;
scanf("%d", &queries);
char town_name[MAX_STRING_LENGTH];
while (queries--) {
    int type;
    scanf("%d", &type);
    switch (type) {
    case 1:
        scanf("%s", town_name);
        town* t = find_town(towns, towns_count, town_name);
        print_all_packages(*t);
        break;
    case 2:
        scanf("%s", town_name);
        town* source = find_town(towns, towns_count, town_name);
        int source_index;
        scanf("%d", &source_index);
        scanf("%s", town_name);
        town* target = find_town(towns, towns_count, town_name);
        int target_index;
        scanf("%d", &target_index);
        send_all_acceptable_packages(source, source_index, target, target_index);
        break;
    }
}

```



```
    send_all_acceptable_packages(source, source_index, target, target_index);  
    break;  
case 3:  
    printf("Town with the most number of packages is %s\n", town_with_most_packages(town  
towns_count).name);  
    break;  
}  
}  
return 0;
```

Congratulations

You solved this challenge. Would you like to challenge your friends?  

Next Challenge

✓ Test case 0

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

town with the most number of packages is A

A:

0:

a

b

1:

c

e

f

h

B:

0:

d